Assignment

Algorithms & Data Structures
Artificial Intelligence

Way-points

Lecturer: Dr David McLean

Muhammad Zahid
25925501

May 2026

1 Introduction

Data storage and data search require a key fundamental aspect: Data Structure. Arrays, Linked-lists(L-L), and binary Trees serve as fundamentals for data structures [Nasare and Wanjari, 2025]. The main problem faced by search algorithms such as way-point search is the speed of storage access, which is a limitation Liu [2025].

Traditional search algorithms suffer from increased storage access times in big data applications, reducing search performance Delimpaltadakis et al. [2025]. Depending on the size of the stored data modification, editing, viewing, inserting, and deleting operations can become major challenges Liu [2025]. L-L, arrays, and Binary trees are solutions to that problem; each of these methods holds limitations as well as advantages [Nasare and Wanjari, 2025].

Here such methods are compared against each other. practices followed for creating a way-point search program will be discussed, with complexity analysis.

2 Data Structure Comparison

For efficient data management appropriate data structures selection is essential. The selection of these different types of structures depends on the type usage required as they offer different trade-offs in access, deletion, insertion and memory usage. Modern approaches for large-scale systems continue the research on hybrid data-structure. But we will be discussing the fundamental methodologies; arrays, linked lists, and binary search trees Sarevska [2024].

In case of this assessment way-point, we first read the way-points from the file using a class called **Waypoint**. The way-points class divides the information from the file into their respective categories.

```
(string name, string code, string latitude, string longitude, int altitude, string description)
```

From here the way-points can further be stored in arrays, L-L, and Binary trees. Further more these stored Way-points can be used to create Routes similar to the concept of navigation via GPS.

2.1 Arrays

Arrays are an efficient method for memory allocation and data access. They have a fixed size set at the initial declaration. This makes them difficult in terms of dynamic data handling, such as the insertion or removal of data in the middle. This can only be tackled through memory relocation or by shifting large portions of the array [Nasare and Wanjari, 2025].

For our navigation system we use arrays to store our way-points, this is similar to the concept of Stack and Queue. We create a private instance array called **Waypoint**. This can be used to store an array of way-points which can be accessed through public properties.

Similar to the concept of Way-point storage in an array, we use same concept for Route storage. The **RouteArray** class stores all the routes in an array with all the information of Way-points stored in a Route from the start of destination to the end.

2.2 Linked Lists

Connected via pointers L-L allows for dynamic insertion and deletion making them extremely flexible. Similar to arrays they have a linear data structure. Unlike arrays they have memory nodes, each of which holds a pointer to the next node, creating a chain like structure [Nasare and Wanjari, 2025].

For the purpose of constructing routes, easy access to the stored Way-point objects is essential. This will allow efficiency in deleting, adding or inserting Way-point data in storage. L-L make this easy, since it consists of memory nodes with a pointer, giving easy access to each way-point node.

This means we need two internal classes one for creation of nodes and another for storing such nodes in a list. Dubbed as **RouteNode** and **Route** respectively, each class node requires data; a way-point, a reference to the next node and their properties. The Route class will contain a name, number of way-points, a **RouteNode** and multiple functions to add, delete or display the Routes.

2.3 Binary Trees

Binary Trees are tree like structures with many applications, such as data storage and retrieval. They are also popular in search algorithms, sorting, and graph algorithms. Although binary search trees provide faster search performance, node deletion is structurally more complex because replacement nodes must be identified and sub-tree links reorganized. Sarevska [2024].

Experimental comparisons show the efficiency of binary search trees. In one study using 20 randomly shuffled elements repeated over 1000 trials, binary search trees required approximately 5 programming steps for search and delete operations, compared to 9 steps for linked lists, demonstrating the advantage of hierarchical search Sarevska [2024].

Although in our assessment binary tree was not used, binary trees are an excellent method that can be used in navigation system. Their ability for efficiency in reduced search cost make them more suitable for larger way-point datasets.

2.4 Evaluation

Arrays are efficient at searching for elements in a dataset; L-L, on the other hand, can easily add or delete stored elements. Binary Trees are as efficient in performing binary search as a stored array, while their performance in the deletion of an element is comparable to L-L. The performance of these can be confirmed by the following research paper Sarevska [2024]. These performance analysis align with the provided referenced study.

3 Complexity Analysis

The most complex dat

3.1 Method: RemoveWayPoint(string name)

```
public void RemoveWayPoint(string name)
{
    RouteNode currentnode = route_wps;              //1
    if (route_wps == null)                          //1
    {
        Console.WriteLine("Route is empty...");     //1
        return;                                     //1
    }
    if (route_wps.Waypoint.Name == name)            //1
    {
        route_wps = route_wps.Next;                 //1
        noOfWaypoints--;                            //1
        return;                                     //1
    }
    while (currentnode.Next != null)                //n+1
    {
        if (name.CompareTo(...) == 0)               //n
        {
            currentnode.Next = currentnode.Next.Next; //n
            noOfWaypoints--;                        //n
        }
        else { currentnode = currentnode.Next; }    //n
    }
}
```

3.2 Complexity Calculation

1. **Initial Assignment**: $T_1 = 1$
2. **Null Check**: $T_2 = 1$
3. **First Node Match Check**: $T_3 = 1$
4. **While Loop**: $T_4 = n + 1$ (condition checks) $+3n$ (loop body operations)

Total operations: $T(n) = 1 + 1 + 1 + (n + 1) + 3n = 4n + 4$
Simplifying: $T(n) = 4n + 4 \approx O(n)$

3.3 Case Analysis

Case	Scenario	Operations	Complexity
Best Case 1	Route is empty (null)	$1 + 1 + 1 + 1 = 4$	$O(1)$
Best Case 2	Delete first waypoint	$1 + 1 + 1 + 1 + 1 + 1 = 6$	$O(1)$
Average Case	Delete middle waypoint	$\approx 2n + 5$	$O(n)$
Worst Case	Delete last waypoint or not found	$4n + 9$	$O(n)$

Table 1: Time Complexity Cases for RemoveWayPoint Method

3.3.1 Detailed Case Breakdown

Case 1: Best Case - Empty Route (O(1)) When the route is empty:

```
if (route_wps == null)
{
    Console.WriteLine("Route is empty...");
    return;
}
```

Operations: $T = 4$ constant operations
Time Complexity: $O(1)$

Case 2: Best Case - Delete First Way-point (O(1)) When the first way-point matches the target name, the method returns immediately without entering the while loop:

```
if (route_wps.Waypoint.Name == name)
{
    route_wps = route_wps.Next;
    noOfWaypoints--;
    return;
}
```

Operations: $T = 6$ constant operations
Time Complexity: $O(1)$

Case 3: Average Case - Delete Middle Way-point (O(n)) When the target way-point is located in the middle of the linked list. The loop executes approximately $n/2$ times on average.
Operations: $T = 4 + (n/2 + 1) + 3(n/2) = 2n + 5$
Time Complexity: $O(n)$

Case 4: Worst Case - Delete Last or Non-existent Way-point (O(n)) When the target way-point is at the end of the list or does not exist, the loop must traverse the entire list. The loop executes n times, with condition checked $n + 1$ times.
Operations: $T = 4 + (n + 1) + 3n = 4n + 5$
Time Complexity: $O(n)$

3.4 Overall Time Complexity

$$T(n) = 4n + 4 \tag{1}$$

Dropping constant coefficients and lower-order terms:

$$O(n) \tag{2}$$

Where n is the number of way-points in the route.

References

Giannis Delimpaltadakis, Pol Mestres, Jorge Cortés, and W. P. M. H. Heemels. Feedback optimization with state constraints through control barrier functions. pages 7234–7239, Dec 2025. doi: 10.1109/CDC57313.2025.11312283. URL `http://dx.doi.org/10.1109/CDC57313.2025.11312283`.

S. Liu. Linked array tree: A constant-time search structure for big data. *arXiv preprint arXiv:2504.00828*, 2025. Available at: https://arxiv.org/abs/2504.00828.

Sarveshkumar Nasare and Sunil Wanjari. Linked-arrays: A mathematical approach. *International Journal on Advanced Computer Theory and Engineering*, 14(2):9–12, 2025. doi: 10.65521/ijacte.v14i2.1706.

Maja Sarevska. Array, linked list, and binary search tree comparison. *WSEAS Transactions on Computers*, 23:144–148, 2024. doi: 10.37394/23205.2024.23.12.